

Nama :Elsa Marthalinda

NIM :254107020204

Kelas :TI – 1G

LAPORAN JOBHSEET 11

2.1 Pembuatan Single Linked List

Kode Program Mahasiswa7.java;

```
package Pertemuan11;

public class Mahasiswa7 {

    String nim;

    String nama;

    String kelas;

    double ipk;

    Mahasiswa7() {

    }

    Mahasiswa7(String nm, String name, String kls, double ip) {

        nim = nm;

        nama = name;

        kelas = kls;

        ipk = ip;

    }

    void tampilInformasi() {

        System.out.println(nama + "\t" + nim + "\t" + kelas + "\t" + ipk);

    }

}
```

Kode Program Node7.java;

```
package Pertemuan11;

public class Node7 {
```

```
Mahasiswa7 data;
```

```
Node7 next;
```

```
public Node7(Mahasiswa7 data, Node7 next) {  
    this.data = data;  
    this.next = next;  
}  
}
```

Kode Program SingleLinkedList7.java;

```
package Pertemuan11;
```

```
public class SingleLinkedList7 {
```

```
    Node7 head;
```

```
    Node7 tail;
```

```
    boolean isEmpty() {  
        return (head == null);  
    }
```

```
    public void print() {  
        if (!isEmpty()) {  
            Node7 tmp = head;  
            System.out.println("Isi Linked List:\t");  
            while (tmp != null) {  
                tmp.data.tampilInformasi();  
                tmp = tmp.next;  
            }  
            System.out.println("");  
        } else {  
            System.out.println("Linked list kosong");  
        }  
    }  
}
```

```

public void addFirst(Mahasiswa7 input) {
    Node7 ndInput = new Node7(input, null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        ndInput.next = head;
        head = ndInput;
    }
}

```

```

public void addLast(Mahasiswa7 input) {
    Node7 ndInput = new Node7(input, null);
    if (isEmpty()) {
        head = ndInput;
        tail = ndInput;
    } else {
        tail.next = ndInput;
        tail = ndInput;
    }
}

```

```

public void insertAfter(String key, Mahasiswa7 input) {
    Node7 ndInput = new Node7(input, null);
    Node7 temp = head;
    do {
        if (temp.data.nama.equalsIgnoreCase(key)) {
            ndInput.next = temp.next;
            temp.next = ndInput;
            if (ndInput.next == null) {
                tail = ndInput;
            }
            break;
        }
    }
}

```

```

        temp = temp.next;
    } while (temp != null);
}

public void insertAt(int index, Mahasiswa7 input) {
    if (index < 0) {
        System.out.println("Index salah");
    } else if (index == 0) {
        addFirst(input);
    } else {
        Node7 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = new Node7(input, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}
}

```

Kode Program SLLMain7.java;

```

package Pertemuan11;

public class SLLMain7 {
    public static void main(String[] args) {
        SingleLinkedList7 sll = new SingleLinkedList7();

        Mahasiswa7 mhs1 = new Mahasiswa7("24212200", "Alvaro","1A", 4.0);
        Mahasiswa7 mhs2 = new Mahasiswa7("23212201", "Bimon","2B", 3.8);
        Mahasiswa7 mhs3 = new Mahasiswa7("22212202", "Cintia","3C", 3.5);
        Mahasiswa7 mhs4 = new Mahasiswa7("21212203", "Dirga","4D", 3.6);
    }
}

```

```

        sll.print();
        sll.addFirst(mhs4);
        sll.print();
        sll.addLast(mhs1);
        sll.print();
        sll.insertAfter("Dirga", mhs3);
        sll.insertAt(2, mhs2);
        sll.print();
    }
}

```

Hasil Output:

```

Linked list kosong
Isi Linked List:
Dirga  21212203      4D      3.6

Isi Linked List:
Dirga  21212203      4D      3.6
Alvaro 24212200      1A      4.0

Isi Linked List:
Dirga  21212203      4D      3.6
Cintia 22212202      3C      3.5
Bimon  23212201      2B      3.8
Alvaro 24212200      1A      4.0

```

2.1.2 Pertanyaan

1. Mengapa hasil compile kode program di baris pertama menghasilkan “Linked List Kosong”?
 - Pada awal program, linked list masih kosong karena belum ada data yang ditambahkan, sehingga head bernilai null. Saat method print() dipanggil, kondisi tersebut terdeteksi oleh isEmpty(), sehingga muncul output “Linked list kosong”.
2. Jelaskan kegunaan variable temp secara umum pada setiap method!
 - Variabel temp digunakan sebagai penunjuk sementara untuk menelusuri linked list dari head ke node berikutnya. Tujuannya agar proses traversal, pencarian, atau penyisipan data dapat dilakukan tanpa mengubah struktur asli linked list. Dengan temp, data tetap aman karena head tidak ikut berubah.
3. Lakukan modifikasi agar data dapat ditambahkan dari keyboard!
 - Modifikasi SLLMain7.java;

```

package Pertemuan11;
import java.util.Scanner;
public class SLLMain7 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SingleLinkedList7 sll = new SingleLinkedList7();

        System.out.print("Masukkan jumlah data: ");
        int n = sc.nextInt();
        sc.nextLine();
        for (int i = 0; i < n; i++) {
            System.out.println("Data mahasiswa ke-" + (i + 1));
            System.out.print("NIM\t\t: ");
            String nim = sc.nextLine();

            System.out.print("Nama\t\t: ");
            String nama = sc.nextLine();

            System.out.print("Kelas\t\t: ");
            String kelas = sc.nextLine();

            System.out.print("IPK\t\t: ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            Mahasiswa7 mhs = new Mahasiswa7(nim, nama, kelas,
ipk);

            sll.addLast(mhs);
        }
        System.out.println("\nData Linked List:");
        sll.print();
    }
}

```

Masukkan jumlah data: 4

Data mahasiswa ke-1

NIM : 24212200

Nama : Alvaro

Kelas : 1A

IPK : 4,0

Data mahasiswa ke-2

NIM : 23212201

Nama : Bimon

Kelas : 2B

IPK : 3,8

Data mahasiswa ke-3

NIM : 22212202

Nama : Cintia

Kelas : 3C

IPK : 3,5

Data mahasiswa ke-4

NIM : 21212203

Nama : Dirga

Kelas : 4D

IPK : 3,6

Data Linked List:

Isi Linked List:

Alvaro 24212200 1A 4.0

Bimon 23212201 2B 3.8

Cintia 22212202 3C 3.5

Dirga 21212203 4D 3.6

2.2 Modifikasi Elemen pada Single Linked List

Modifikasi Kode Program SingleLinked7.java;

```
package Pertemuan11;
public class SingleLinkedList7 {
    Node7 head;
    Node7 tail;

    boolean isEmpty() {
        return (head == null);
    }

    public void print() {
        if (!isEmpty()) {
            Node7 tmp = head;
            System.out.println("Isi Linked List:\t");
            while (tmp != null) {
                tmp.data.tampilInformasi();
                tmp = tmp.next;
            }
            System.out.println("");
        } else {
            System.out.println("Linked list kosong");
        }
    }

    public void addFirst(Mahasiswa7 input) {
        Node7 ndInput = new Node7(input, null);
        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            ndInput.next = head;
            head = ndInput;
        }
    }

    public void addLast(Mahasiswa7 input) {
        Node7 ndInput = new Node7(input, null);
        if (isEmpty()) {
            head = ndInput;
            tail = ndInput;
        } else {
            tail.next = ndInput;
            tail = ndInput;
        }
    }

    public void insertAfter(String key, Mahasiswa7 input) {
        Node7 ndInput = new Node7(input, null);
        Node7 temp = head;
        do {
            if (temp.data.nama.equalsIgnoreCase(key)) {
                ndInput.next = temp.next;
                temp.next = ndInput;
                if (ndInput.next == null) {

```

```

        tail = ndInput;
    }
    break;
}
temp = temp.next;
} while (temp != null);
}

public void insertAt(int index, Mahasiswa7 input) {
    if (index < 0) {
        System.out.println("Index salah");
    } else if (index == 0) {
        addFirst(input);
    } else {
        Node7 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = new Node7(input, temp.next);
        if (temp.next.next == null) {
            tail = temp.next;
        }
    }
}

public void getData(int index) {
    Node7 temp = head;
    for (int i = 0; i < index; i++) {
        temp = temp.next;
    }
    temp.data.tampilInformasi();
}

public int indexOf(String key) {
    Node7 tmp = head;
    int index = 0;
    while (tmp != null && !tmp.data.nama.equalsIgnoreCase(key)) {
        tmp = tmp.next;
        index++;
    }

    if (tmp == null) {
        return -1;
    } else {
        return index;
    }
}

public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked list masih kosong, tidak dapat
dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
    }
}

```



```

    }
}

public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked list masih kosong, tidak dapat
dihapus!");
    } else if (head == tail) {
        head = tail = null;
    } else {
        Node7 temp = head;
        while (temp.next != tail) {
            temp = temp.next;
        }
        temp.next = null;
        tail = temp;
    }
}

public void remove(String key) {
    if (isEmpty()) {
        System.out.println("Linked list masih kosong, tidak dapat
dihapus!");
    } else {
        Node7 temp = head;
        while (temp != null) {
            if ((temp.data.nama.equalsIgnoreCase(key)) && temp == head) {
                this.removeFirst();
                break;
            } else if (temp.data.nama.equalsIgnoreCase(key)) {
                temp.next = temp.next.next;
                if (temp.next == null) {
                    tail = temp;
                }
                break;
            }
            temp = temp.next;
        }
    }
}

public void removeAt(int index) {
    if (index == 0) {
        removeFirst();
    } else {
        Node7 temp = head;
        for (int i = 0; i < index - 1; i++) {
            temp = temp.next;
        }
        temp.next = temp.next.next;
        if (temp.next == null) {
            tail = temp;
        }
    }
}
}

```

Modifikasi Kode Program SLLMain7.java;

```
package Pertemuan11;
import java.util.Scanner;
public class SLLMain7 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        SingleLinkedList7 sll = new SingleLinkedList7();

        System.out.print("Masukkan jumlah data: ");
        int n = sc.nextInt();
        sc.nextLine();
        for (int i = 0; i < n; i++) {
            System.out.println("Data mahasiswa ke-" + (i + 1));
            System.out.print("NIM\t: ");
            String nim = sc.nextLine();

            System.out.print("Nama\t: ");
            String nama = sc.nextLine();

            System.out.print("Kelas\t: ");
            String kelas = sc.nextLine();

            System.out.print("IPK\t: ");
            double ipk = sc.nextDouble();
            sc.nextLine();

            Mahasiswa7 mhs = new Mahasiswa7(nim, nama, kelas, ipk);
            sll.addLast(mhs);
        }
        System.out.println("\nData Linked List:");
        sll.print();

        System.out.println("data index 1 : ");
        sll.getData(1);

        System.out.println("data mahasiswa an Bimon berada pada index : " + sll.indexOf("Bimon"));
        System.out.println();

        sll.removeFirst();
        sll.removeLast();
        sll.print();
        sll.removeAt(0);
        sll.print();
    }
}
```

Hasil Output:

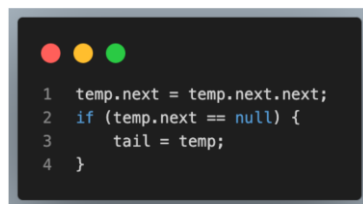
```
data index 1 :
Cintia 22212202      3C      3.5
data mahasiswa an Bimon berada pada index : 2
```

```
Isi Linked List:
Cintia 22212202      3C      3.5
Bimon  23212201      2B      3.8
```

```
Isi Linked List:
Bimon  23212201      2B      3.8
```

2.2.3 Pertanyaan

1. Mengapa digunakan keyword break pada fungsi remove? Jelaskan!
 - Pada fungsi remove, keyword break digunakan untuk menghentikan proses perulangan setelah data yang dicari berhasil ditemukan dan dihapus. Karena setelah node dihapus, struktur linked list sudah berubah sehingga tidak perlu melanjutkan pencarian ke node berikutnya. Selain itu, penggunaan break juga mencegah terjadinya proses yang tidak perlu dan menghindari potensi kesalahan dalam manipulasi data.
2. Jelaskan kegunaan kode dibawah pada method remove



```
1 temp.next = temp.next.next;
2 if (temp.next == null) {
3     tail = temp;
4 }
```

- Baris `temp.next = temp.next.next;` berfungsi untuk melewati node yang akan dihapus, sehingga node tersebut tidak lagi terhubung dalam linked list. Node sebelumnya langsung dihubungkan ke node setelahnya. Selanjutnya, kondisi `if (temp.next == null)` digunakan untuk mengecek apakah node yang dihapus adalah node terakhir. Jika benar, maka `tail` diperbarui menjadi `temp`, karena sekarang `temp` menjadi node terakhir dalam linked list.

TUGAS

Kode Program DataMahasiswa7.java;

```
package Pertemuan11;
class DataMahasiswa7 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public DataMahasiswa7(String nim, String nama, String kelas, double
ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampil() {
        System.out.println(nama + "\t" + nim + "\t" + kelas + "\t" + ipk);
    }
}
```

```
}  
}
```

Kode Program NodeQL7.java;

```
package Pertemuan11;  
  
public class NodeQL7 {  
    DataMahasiswa7 data;  
    NodeQL7 next;  
  
    public NodeQL7(DataMahasiswa7 data) {  
        this.data = data;  
        this.next = null;  
    }  
}
```

Kode Program QueueLinkedList7.java;

```
package Pertemuan11;  
public class QueueLinkedList7 {  
    NodeQL7 front, rear;  
    int size;  
  
    public QueueLinkedList7() {  
        front = rear = null;  
        size = 0;  
    }  
  
    public boolean isEmpty() {  
        return front == null;  
    }  
  
    public void enqueue(DataMahasiswa7 data) {  
        NodeQL7 newNode = new NodeQL7(data);  
  
        if (isEmpty()) {  
            front = rear = newNode;  
        } else {  
            rear.next = newNode;  
            rear = newNode;  
        }  
  
        size++;  
        System.out.println("Mahasiswa berhasil ditambahkan ke antrian.");  
    }  
  
    public void dequeue() {  
        if (isEmpty()) {  
            System.out.println("Antrian kosong!");  
            return;  
        }  
    }  
}
```

```

        System.out.println("Memanggil antrian:");
        front.data.tampil();

        front = front.next;
        size--;

        if (front == null) {
            rear = null;
        }
    }

    public void peekFront() {
        if (!isEmpty()) {
            System.out.println("Antrian terdepan:");
            front.data.tampil();
        } else {
            System.out.println("Antrian kosong!");
        }
    }

    public void peekRear() {
        if (!isEmpty()) {
            System.out.println("Antrian terakhir:");
            rear.data.tampil();
        } else {
            System.out.println("Antrian kosong!");
        }
    }

    public void printQueue() {
        if (isEmpty()) {
            System.out.println("Antrian kosong!");
            return;
        }

        NodeQL7 temp = front;
        System.out.println("Daftar antrian:");
        while (temp != null) {
            temp.data.tampil();
            System.out.println("-----");
            temp = temp.next;
        }
    }

    public void clear() {
        front = rear = null;
        size = 0;
        System.out.println("Antrian berhasil dikosongkan.");
    }

    public void jumlahAntrian() {
        System.out.println("Jumlah mahasiswa dalam antrian: " + size);
    }
}

```

Kode Program QLLMain7.java;

```
package Pertemuan11;
import java.util.Scanner;
public class QLLMain7 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        QueueLinkedList7 queue = new QueueLinkedList7();

        int pilih;
        do {
            System.out.println("\n=== MENU ANTRIAN KEMAHASISWAAN ===");
            System.out.println("1. Menambahkan Antrian");
            System.out.println("2. Memanggil Antrian");
            System.out.println("3. Tampilkan Semua Antrian");
            System.out.println("4. Tampil Antrian Terdepan & Terakhir");
            System.out.println("5. Jumlah Antrian");
            System.out.println("6. Kosongkan Antrian");
            System.out.println("0. Keluar");
            System.out.print("Pilih menu: ");
            pilih = sc.nextInt();
            sc.nextLine();

            switch (pilih) {
                case 1:
                    System.out.print("NIM   : ");
                    String nim = sc.nextLine();
                    System.out.print("Nama   : ");
                    String nama = sc.nextLine();
                    System.out.print("Kelas : ");
                    String kelas = sc.nextLine();
                    System.out.print("IPK    : ");
                    double ipk = sc.nextDouble();
                    sc.nextLine();

                    DataMahasiswa7 mhs = new DataMahasiswa7(nim, nama,
kelas, ipk);

                    queue.enqueue(mhs);
                    break;

                case 2:
                    queue.dequeue();
                    break;

                case 3:
                    queue.printQueue();
                    break;

                case 4:
                    queue.peekFront();
                    queue.peekRear();
                    break;

                case 5:
                    queue.jumlahAntrian();
                    break;
            }
        } while (pilih != 0);
    }
}
```

```

        case 6:
            queue.clear();
            break;

        case 0:
            System.out.println("Terima kasih!");
            break;

        default:
            System.out.println("Menu tidak valid!");
    }

    } while (pilih != 0);
}

```

Hasil Output:

=== MENU ANTRIAN KEMAHASISWAAN ===

1. Menambahkan Antrian
2. Memanggil Antrian
3. Tampilkan Semua Antrian
4. Tampil Antrian Terdepan & Terakhir
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 1

NIM : 24212200

Nama : Alvaro

Kelas : 1A

IPK : 4,0

Mahasiswa berhasil ditambahkan ke antrian.

=== MENU ANTRIAN KEMAHASISWAAN ===

1. Menambahkan Antrian
2. Memanggil Antrian
3. Tampilkan Semua Antrian
4. Tampil Antrian Terdepan & Terakhir
5. Jumlah Antrian

6. Kosongkan Antrian

0. Keluar

Pilih menu: 1

NIM : 23212201

Nama : Bimon

Kelas : 2B

IPK : 3,8

Mahasiswa berhasil ditambahkan ke antrian.

=== MENU ANTRIAN KEMAHASISWAAN ===

1. Menambahkan Antrian

2. Memanggil Antrian

3. Tampilkan Semua Antrian

4. Tampil Antrian Terdepan & Terakhir

5. Jumlah Antrian

6. Kosongkan Antrian

0. Keluar

Pilih menu: 1

NIM : 22212202

Nama : Cintia

Kelas : 3C

IPK : 3,5

Mahasiswa berhasil ditambahkan ke antrian.

=== MENU ANTRIAN KEMAHASISWAAN ===

1. Menambahkan Antrian

2. Memanggil Antrian

3. Tampilkan Semua Antrian

4. Tampil Antrian Terdepan & Terakhir

5. Jumlah Antrian

6. Kosongkan Antrian

0. Keluar

Pilih menu: 1

NIM : 21212203

Nama : Dirga

Kelas : 4D

IPK : 3,6

Mahasiswa berhasil ditambahkan ke antrian.

=== MENU ANTRIAN KEMAHASISWAAN ===

1. Menambahkan Antrian
2. Memanggil Antrian
3. Tampilkan Semua Antrian
4. Tampil Antrian Terdepan & Terakhir
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 2

Memanggil antrian:

Alvaro 24212200 1A 4.0

=== MENU ANTRIAN KEMAHASISWAAN ===

1. Menambahkan Antrian
2. Memanggil Antrian
3. Tampilkan Semua Antrian
4. Tampil Antrian Terdepan & Terakhir
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 3

Daftar antrian:

Bimon 23212201 2B 3.8

Cintia 22212202 3C 3.5

Dirga 21212203 4D 3.6

=== MENU ANTRIAN KEMAHASISWAAN ===

1. Menambahkan Antrian
2. Memanggil Antrian
3. Tampilkan Semua Antrian
4. Tampil Antrian Terdepan & Terakhir
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 4

Antrian terdepan:

Bimon 23212201 2B 3.8

Antrian terakhir:

Dirga 21212203 4D 3.6

=== MENU ANTRIAN KEMAHASISWAAN ===

1. Menambahkan Antrian
2. Memanggil Antrian
3. Tampilkan Semua Antrian
4. Tampil Antrian Terdepan & Terakhir
5. Jumlah Antrian
6. Kosongkan Antrian
0. Keluar

Pilih menu: 5

Jumlah mahasiswa dalam antrian: 3

=== MENU ANTRIAN KEMAHASISWAAN ===

1. Menambahkan Antrian
2. Memanggil Antrian
3. Tampilkan Semua Antrian
4. Tampil Antrian Terdepan & Terakhir
5. Jumlah Antrian

6. Kosongkan Antrian

0. Keluar

Pilih menu: 6

Antrian berhasil dikosongkan.

=== MENU ANTRIAN KEMAHASISWAAN ===

1. Menambahkan Antrian

2. Memanggil Antrian

3. Tampilkan Semua Antrian

4. Tampil Antrian Terdepan & Terakhir

5. Jumlah Antrian

6. Kosongkan Antrian

0. Keluar

Pilih menu: 3

Antrian kosong!

=== MENU ANTRIAN KEMAHASISWAAN ===

1. Menambahkan Antrian

2. Memanggil Antrian

3. Tampilkan Semua Antrian

4. Tampil Antrian Terdepan & Terakhir

5. Jumlah Antrian

6. Kosongkan Antrian

0. Keluar

Pilih menu: 0

Terima kasih!